

Views

08_etapa2_views.sql — três consultas guardadas no banco

DROP e CREATE, não CREATE OR REPLACE

As três views são recriadas com DROP VIEW IF EXISTS seguido de CREATE VIEW. CREATE OR REPLACE VIEW só aceita manter a mesma lista de colunas, com os mesmos nomes e tipos, na mesma ordem. Qualquer ajuste na consulta que acrescente ou renomeie coluna falharia, e é exatamente isso que acontece enquanto a consulta está sendo escrita. O DROP não usa CASCADE, para que a remoção falhe em voz alta se algum objeto passar a depender da view.

vw_pacientes_internados

Pacientes internados no momento. O enunciado define o critério como data_hora_saida nula na internação mais recente, e essa formulação é mais restrita do que parece.

Duas leituras diferentes

"Existe internação sem saída" e "a internação mais recente está sem saída" dão resultados diferentes. Um paciente com uma internação antiga que ninguém encerrou apareceria como internado para sempre pela primeira leitura. A view implementa a segunda.

Estrutura da view

```
WITH ultima_internacao AS (  
    SELECT DISTINCT ON (i.id_paciente)  
        i.id_internacao, i.id_paciente, i.id_unidade,  
        i.data_hora_entrada, i.data_hora_saida, i.motivo  
    FROM Internacao i  
    ORDER BY i.id_paciente, i.data_hora_entrada DESC, i.id_internacao DESC  
)  
SELECT ...  
    FROM ultima_internacao ui  
    WHERE ui.data_hora_saida IS NULL;
```

DISTINCT ON é uma extensão do PostgreSQL. Combinada com o ORDER BY acima, devolve uma linha por paciente, a de entrada mais recente. O desempate por id_internacao cobre duas entradas no mesmo instante. Em SQL padrão o mesmo efeito sairia de uma função de janela com row_number, com mais texto.

A view também calcula tempo_internado a partir de CURRENT_TIMESTAMP, então o valor muda a cada consulta.

Resultado esperado com o seed

Paciente	Situação no seed	Aparece
Carla Mendes	internada na UTI em 22/07, sem alta	sim
Ana Souza	internação de maio encerrada e uma de 24/07 aberta	sim
Elisa Rocha	internada na Enfermaria A em 25/07, sem alta	sim
Bruno Lima	internação única, com alta em 25/06	não
Diego Ferreira	internação mais recente encerrada em 12/07	não

O caso de Ana Souza é o que distingue as duas leituras. Ela tem histórico e uma internação aberta, e aparece por causa da mais recente.

`vw_residentes_sem_supervisor`

Plantões cujo preceptor responsável não tem titulação de doutor. Devolve uma linha por escala, com residente, unidade, dia, turno, preceptor e o motivo da inclusão.

O enunciado menciona também residentes sem supervisão ativa. No esquema atual essa situação não é possível: `escala.id_preceptor` é NOT NULL com chave estrangeira, então toda escala tem preceptor. A view usa LEFT JOIN e testa o nulo de todo modo, o que documenta que o caso foi considerado e deixa a consulta pronta se a coluna se tornar opcional. A coluna motivo distingue os dois casos.

A comparação usa `lower(pre.titulacao)` porque `titulacao` é texto livre. O seed grava "doutor", mas nada impede que a API receba "Doutor".

Com o seed, a view devolve quatro linhas. Karina Duarte na terça à tarde, supervisionada por um mestre; Nathan Ribeiro na sexta pela manhã, com um especialista; e Olivia Prado duas vezes, no sábado à tarde e na sexta à tarde, as duas com mestres. Os plantões dos preceptores 6 e 8, que são doutores, ficam de fora.

`vw_estatisticas_atendimentos_mensal`

Agregação por mês e unidade, com total de atendimentos, duração média, menor e maior duração, e os procedimentos mais frequentes.

O ranking dentro de cada grupo

Total e média saem de um GROUP BY comum. "Procedimentos mais comuns" não sai: é um ranking dentro de cada mês e unidade, e não um valor agregado único. A view resolve isso com três CTE.

CTE	Papel
base	atendimentos com unidade, já com o mês truncado
totais	contagem, média, mínimo e máximo por mês e unidade
contagem	procedimentos numerados por frequência, com ROW_NUMBER

Dentro da CTE contagem, que tem GROUP BY

```
ROW_NUMBER() OVER (PARTITION BY b.mes, b.id_unidade
                    ORDER BY COUNT(*) DESC, pc.nome) AS posicao
```

Usar COUNT(*) dentro do OVER de uma consulta que já tem GROUP BY funciona porque funções de janela são avaliadas depois da agregação. O SELECT final junta os três primeiros colocados num texto com string_agg.

Duas escolhas de contagem

`total_atendimentos` vem da CTE base, não da junção com `procedimento_realizado`. Um atendimento sem procedimento registrado ainda conta no total, e nesse caso `procedimentos_mais_comuns` fica nulo. Se o total viesse da junção, atendimentos sem procedimento desapareceriam e os com vários seriam contados mais de uma vez.

Atendimentos sem unidade ficam de fora da view inteira, o que inclui os criados pelas rotas em SQL puro da Etapa 1, que não informam o campo.

Resultado esperado com o seed

Mês	Unidade	Atend.	Média (min)	Mais comuns
2026-07	Ambulatorio	2	37,5	Coleta, Curativo, ECG
2026-07	Enfermaria A	3	45,0	Aplicacao, Curativo, Drenagem
2026-07	Pronto-Socorro	3	26,7	Acesso venoso, Coleta, ECG
2026-07	UTI Adulto	5	52,0	Coleta (2), Intubacao (2), Puncao (2)
2026-06	Enfermaria A	1	30,0	Sutura simples
2026-06	Pronto-Socorro	1	45,0	Aplicacao, Coleta

Os nomes dos procedimentos aparecem abreviados nesta tabela. A view devolve o nome completo seguido da contagem entre parênteses. Quando há empate, a ordem é alfabética.

Verificação

A seção 2 do 10_etapa2_verificacao.sql consulta as três views e traz os resultados esperados em comentário. Pela API, os três endpoints ficam em GET /etapa2/views/pacientes-internados, /residentes-sem-supervisor e /estatisticas-mensais. Na interface, a aba Views mostra as três em cartões separados, cada um com o critério explicado.

A API consulta as views com SELECT direto, sem passar pela ORM. Uma view é somente leitura e não tem chave primária, que é o que o mapa de identidade da sessão usa para acompanhar objetos, então mapeá-la como entidade acrescentaria código sem mudar o resultado.